



Text Clustering and Recommender Systems

IFN647 – Week 11

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

Topic ID	Key topics	Mapping to slides	Activities
1	Cluster Analysis – Unsupervised Learning	2-6	Q2
2	Clustering Methods	7-13	
3	LDA (Model)-Based Clustering and Evaluation	14-18	Q3
4	LLM-Era Clustering	19-22	
5	Recommender Systems	23-30	Q4
6	LLMs for News Recommendation	31-35	
	References	36	

1. Cluster Analysis – Unsupervised Learning

- Clustering is the process of grouping documents (or objects) into clusters so that objects in a cluster are highly similar to each other, but very different from documents in other clusters.
- Cluster analysis is a popular **unsupervised learning method** (i.e., it does not require any training data).
- We usually use a **distance function** to measure the dissimilarities between objects.
 - Note a similarity measure, which typically has a value s from 0 to 1, can be converted into a distance measure by using $1 - s$.
- Evaluating the output of a clustering algorithm can be challenging since clustering is an unsupervised learning, there is often little or no labeled data to use for the purpose of evaluation.
- Text clustering is a fundamental task in natural language processing (NLP), which aims to group similar texts based on their content without prior labeling [6].
- Traditional text clustering methods typically involve transforming textual data into numerical representations and applying clustering algorithms to group similar texts based on these representations.

Information Table

- As mentioned earlier, a training set is user feedback, which can be described as an information table $(\mathbf{D}, \mathbf{T})$, where
 - $\mathbf{D}$ is a set of documents in which each document is a set of terms (may include duplicate terms); and
 - $\mathbf{T} = \{t_1, t_2, \dots, t_n\}$ is a set of selected terms for all documents in $\mathbf{D}$.

<i>Doc</i>	<i>Terms</i>	<i>Rel</i>
d_1	GERMAN VW GERMAN	Yes
d_2	US US ECONOM ESPIONAG	Yes
d_3	US BILL ECONOM ECONOM ESPIONAG ECONOM	Yes
d_4	US ECONOM ESPIONAG BILL	Yes
d_5	GERMAN MAN VW ESPIONAG	Yes
d_6	GERMAN GERMAN MAN VW ESPIONAG	Yes
d_7	GERMAN VW GERMAN VM	No
d_8	US ECONOM	No

An example of an Information Table

$\mathbf{D} = \{d_1, d_2, \dots, d_8\}$

$\mathbf{T} = \{\text{GERMAN, VW, US, ECONOM, SPY, BILL, ESPIONAG, MAN}\}$, where terms are stemmed keywords.

Binary Variables – Bag-Of-Words Features

- Bag-of-words features are numerical representations of text, created by calculating word frequencies, distributions or weights in a document while disregarding word order and grammar.
- The simple way of using bag-of-words features to represent text for clustering is to review words as binary variables, which will support us to clearly understand how to define distance functions for clustering.
- **A binary variable** has only two states: 0 or 1, where
 - 0 denotes the variable is absent and
 - 1 denotes presence.
- The table on the right is the binary information table of the previous example information table, where each term (word) t_i is a binary variable.
 - E.g., t_5 in the binary information table:
 - 1 indicates that d_3 and d_4 include it, while
 - 0 indicates that other documents do not include it.

<i>Doc</i>	t_1	t_2	t_3	t_4	t_5	t_6	t_7	<i>Rel</i>
d_1	1	1	0	0	0	0	0	yes
d_2	0	0	1	1	0	1	0	yes
d_3	0	0	1	1	1	1	0	yes
d_4	0	0	1	1	1	1	0	yes
d_5	1	1	0	0	0	1	1	yes
d_6	1	1	0	0	0	1	1	yes
d_7	1	1	0	0	0	0	0	no
d_8	0	0	1	1	0	0	0	no

Example of a Binary Information Table, where

- GERMAN : t_1
- VW : t_2
- US : t_3
- ECONOM : t_4
- BILL : t_5
- ESPIONAG : t_6
- MAN : t_7

Binary Contingency Table

- A contingency table is a type of table in a matrix format that displays the frequency distribution of the variables.
- We can use it to comparing the difference between two documents d_i and d_j based on their binary information table representation.
- Assume
 - q is the number of variables that equal 1 in both documents d_j and d_i ;
 - t is the number of variables that equal 0 in both documents d_j and d_i ;
 - s is the number of variables that equal 0 in d_i but equal 1 in d_j ; and
 - r is the number of variables that equal 1 in d_i but equal 0 in d_j .
 - $u=q+r+s+t$, the total number of variables.

		Document d_j		
		1	0	Sum
Document d_i	1	q	r	$q+r$
	0	s	t	$s+t$
	Sum	$q+s$	$r+t$	u

Distance Functions

- Simple matching coefficient, usually for symmetric binary variables, that is, no preference on 1 and 0.

$$dis_s(d_i, d_j) = \frac{r + s}{q + r + s + t}$$

- Jaccard coefficient, usually for asymmetric binary variables, that is, 1 (or 0) is more important than 0 (or 1).

$$dis_{Jac}(d_i, d_j) = \frac{r + s}{q + r + s}$$

		Document d_2		
		1	0	Sum
Document d_1	1	0	2	2
	0	3	2	5
	Sum	3	4	7

- $dis_s(d_1, d_2) = 5/7$
- $dis_{Jac}(d_1, d_2) = 5/5 = 1$

2. Clustering Methods

Partitioning Approach

- Given a set of documents (objects), a partitioning approach attempts to construct k partitions (clusters or groups) of objects such that
 - Each partition must contain at least one object; and each object must belong to exactly one partition. It creates an initial partitioning and uses an iterative relocation technique to reshuffle objects to improve the initial partitioning.

Hierarchical Methods, which build a hierarchical classification of objects.

- Agglomerative (also called bottom-up method)
 - It starts with each object that represents a single group. It then merges the smaller groups into larger ones until a terminal condition holds (or all objects in the same group).
- Divisive method (also called top-down method)
 - It starts with all objects in the same group. It then splits the larger groups into smaller ones until a terminal condition holds (or each object is in a separate group).

Clustering Methods cont.

Density-based approach

- Given a set of points in some space, it groups together points that are closely packed (points with many [nearby neighbors](#)), and marks as outliers points that lie alone in low-density regions (those whose nearest neighbors are too far away).
- DBSCAN is one of the most commonly used and cited clustering algorithms

Self-Organizing Maps (SOM)

It maps higher-dimensional data to a low-dimensional (typically two-dimensional) space, preserving the topological structure of the data.

- A map space consists of components called "nodes" or "neurons", which are arranged as a hexagonal or rectangular grid with two dimensions.
- An SOM is a type of artificial neural network but is trained using competitive learning rather than the error-correction learning used by other artificial neural networks.

K-medoids Technique

- It is a partitioning approach, where each partition (cluster) is represented by one of the objects that locate near the centre of the cluster.
- We use it to understand the basic process of clustering and how to evaluate clustering algorithms.
- The basic idea of the k-medoids technique is to find k clusters in n objects by
 - first arbitrarily selecting a representative object (the medoid) for each cluster;
 - for each remaining object
 - classify it to the cluster that it is the most similar;
 - iteratively replaces one of the medoids by one of the non-medoids if **the quality** of the resulting partitioning is improved. Please note medoids are always restricted to be **members** of the data set.

The Square-Error Criterion

- The quality is estimated using a **cost function** which measures the average dissimilarity between objects and their medoids.
- Typically, a square-error criterion is used to describe the cost function, which satisfy

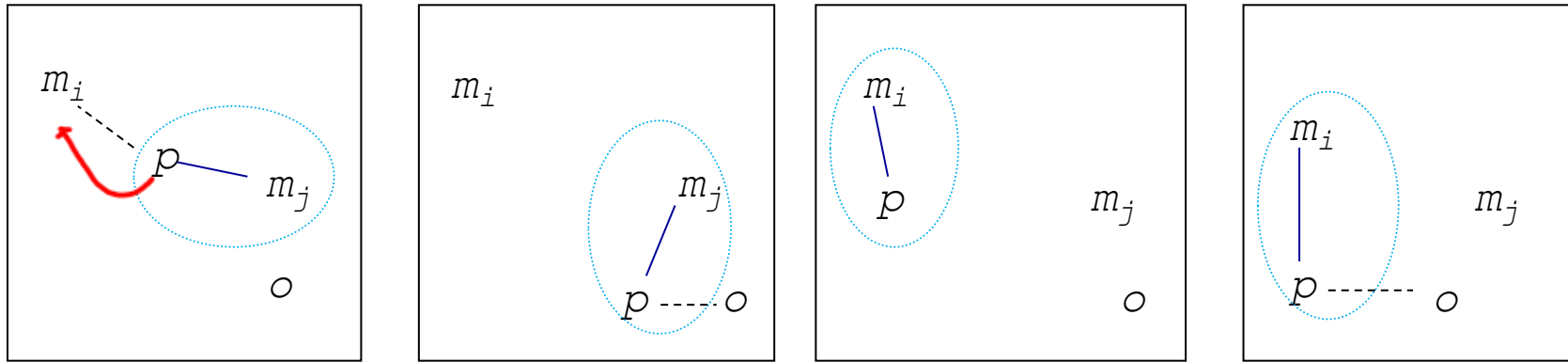
$$E = \sum_{i=1}^k \sum_{p \in C_i} (\text{dis}(p, m_i))^2$$

- where E is the sum of the square-error for all objects, C_i are the clusters and m_i are the medoids ($i = 1, 2, \dots, k$).

Determine New Medoids

- To determine whether a non-medoid object o is a good replacement for a current medoid m_j , we need to discuss all possible cases for all other non-medoid objects p :
 - Case 1: p is reassigned to another medoid m_i
 - p currently belongs to m_j . If m_j is replaced by o as a medoid and p is closest to m_i ($i \neq j$).
 - Case 2: p is reassigned to o
 - p currently belongs to m_j . If m_j is replaced by o as a medoid and p is closest to o .
 - Case 3: the assigned does not change
 - p currently belongs to m_i ($i \neq j$). If m_j is replaced by o as a medoid and p is still closest to m_i .
 - Case 4: p is reassigned to o
 - p currently belongs to m_i ($i \neq j$). If m_j is replaced by o as a medoid and p is closest to o .

Four Cases for Evaluation of a Cost Function



Case 1

Case 2

Case 3

Case 4

- Each time a reassignment occurs, a difference in square-error E is contributed to the cost function.
- The cost function calculates the difference in square-error value if a current medoid is replaced by a non-medoid.
- The **total cost of swapping** (S) is the sum of costs incurred by all non-medoid objects. If the total cost is negative, we swap m_j and o since the square-error value E would be reduced.

CRICOS No.00213J

K-medoids Algorithm

13

Input – the number of clusters k and a database containing n objects.

Output - A set of k clusters that minimizes the sum of the dissimilarities of all the objects to their nearest medoid.

procedure:

- (1) Arbitrarily choose k objects $(m_1, \dots, m_k)$, as the initial medoids;
- (2) Repeat
 - Assign each remaining object to the cluster with the nearest medoid;
 - Randomly select a non-medoid object, o ;
 - Evaluate the total cost, S , of swapping m_j with o ;
 - If $(S < 0)$
 - swap m_j with o and form the new groups of the k medoids;
- (3) Until no change;

- This algorithm attempts to decide k partitions for n documents (objects).
- In step (1), the initial k medoids are determined randomly;
- The algorithm repeatedly tries to get the best medoids for all partitions in step (2). All of the possible pairs of objects are examined.

CRICOS No.00213J

3. LDA (Model)-Based Clustering and Evaluation

- It groups documents by their **topic distributions** learned from **Latent Dirichlet Allocation (LDA)**.
- Instead of clustering on raw words or embeddings, documents are represented as **probability vectors over topics**.
- Model-based clustering is particularly useful in **NLP** because text data has properties that simple distance-based methods (like k-means) struggle to capture.
- LDA is a **generative probabilistic model**:
 - For a given set of documents, each document is a mixture of latent topics.
 - Each topic is a distribution over words.
- After training:
 - Each document d is represented as a topic distribution
$$\theta_d = [p(z_1 | d), p(z_2 | d), \dots, p(z_K | d)], \text{ where}$$
$$z_1, z_2, \dots, z_K \text{ are } K \text{ latent topics found in the set of documents, and}$$
$$K \text{ is a pre-defined parameter.}$$
- We then can use a clustering algorithm to cluster documents by using their topic distributions.

CRICOS No. 002131

Python solution

15

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer
docs = [d1, d2, d3, d4, d5, d6, d7, d8]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)
lda = LatentDirichletAllocation(n_components=3)
lda.fit(X)
# Get document-topic distribution
doc_topics = lda.transform(X)
print("\nDocument-topic distribution:")
print(doc_topics)
```

```
d1 = "GERMANY VW chief wants boost return sales tenfold"
d2 = "USA US Congress moves curb economic spying"
d3 = "US Bill ECONOM ECONOM ESPIONAG ECONOM"
d4 = "US Economic espionage bill clears Senate"
d5 = "GERMANY FOCUS Two men held VW industrial espionage"
d6 = "GERMAN GERMAN MAN VW ESPIONAG"
d7 = "GERMAN VW GERMAN VM"
d8 = "US ECONOM"
```

CRICOS No. 00213J

<i>Doc</i>	<i>Keywords</i>	<i>Rel (label)</i>
d_1	GERMANY VW chief wants boost return sales tenfold	Yes
d_2	USA US Congress moves curb economic spying	Yes
d_3	US Bill ECONOM ECONOM ESPIONAG ECONOM	Yes
d_4	US Economic espionage bill clears Senate	Yes
d_5	GERMANY FOCUS Two men held VW industrial espionage	Yes
d_6	GERMAN GERMAN MAN VW ESPIONAG	Yes
d_7	GERMAN VW GERMAN VM	No
d_8	US ECONOM	No

Document-topic distribution:

<i>topic0</i>	<i>topic 1</i>	<i>topic 2</i>
0.03752991	0.92504463	0.03742546
0.91336138	0.04194987	0.04468876
0.04820144	0.04825992	0.90353864
0.05722475	0.23481157	0.70796369
0.0374915	0.92508548	0.03742302
0.05603147	0.88467869	0.05928984
0.06711399	0.8658654	0.06702061
0.11462151	0.11123325	0.77414523

Python cont.

Topic 0: ['econom', 'us', 'espionag', 'economic', 'bill']
 Topic 1: ['espionage', 'industrial', 'men', 'focus', 'two']
 Topic 2: ['german', 'vw', 'germany', 'wants', 'boost']

```
# Get feature names (words)
feature_names = vectorizer.get_feature_names_out()
# Print top words per topic
n_top_words = 5
for topic_idx, topic in enumerate(lda.components_):
    top_words = [feature_names[i] for i in topic.argsort()[-n_top_words:][::-1]]
    print(f"Topic {topic_idx}: {top_words}")

# Assign clusters (hard clustering)
clusters = doc_topics.argmax(axis=1)
print("\nDocument clusters:")
y_pred = []
for i, cluster in enumerate(clusters):
    print(f"Doc {i} -> Cluster {cluster}")
    y_pred.append(cluster)
print(y_pred)
```

Document clusters:

Doc 0 -> Cluster 1
 Doc 1 -> Cluster 0
 Doc 2 -> Cluster 2
 Doc 3 -> Cluster 2
 Doc 4 -> Cluster 1
 Doc 5 -> Cluster 1
 Doc 6 -> Cluster 1
 Doc 7 -> Cluster 2

[1, 0, 2, 2, 1, 1, 1, 2]

Evaluate the quality of the clustering

- It is challenging to evaluate since clustering is an unsupervised learning algorithm.
- If some labeled data exists, then it is possible to use slightly modified versions of standard IR metrics, such as precision and recall, to evaluate the quality of the clustering.
- Let D be a set of instances (documents) and $\mathbf{C}$ be a set of K clusters produced by a cluster algorithm over D . For each cluster C_i in $\mathbf{C}$, we define a cluster label $label(C_i)$ to be the (human-assigned) class label (e.g., “spam”) associated with the most instances in C_i .

$$label(C_i) = \operatorname{argmax}_{x \in \text{Class}} |\{d \in C_i, label(d) = x\}|$$
- Then we define $MaxClass(C_i)$, which is the set of instances with the cluster label in C_i

$$MaxClass(C_i) = \{d \in C_i, label(d) = label(C_i)\}$$
- The precision for cluster C_i is the fraction of instances in the cluster with $label(C_i)$, and the **cluster precision (purity)** for $\mathbf{C}$ is

$$cluster_{precision} = \frac{\sum_{i=1}^K |MaxClass(C_i)|}{N}$$

where $N = |D|$.

Python function

Indices:

```
[1]
[0 4 5 6]
[2 3 7]
```

Labels:

```
['Yes']
['Yes' 'Yes' 'Yes' 'No']
['Yes' 'Yes' 'No']
```

Most_common_count:

```
1
3
2
```

Purity: 0.75

```
import numpy as np
from collections import Counter
def purity_score(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    total_correct = 0
    for cluster in np.unique(y_pred):
        # indices of points in this cluster
        indices = np.where(y_pred == cluster)[0]
        # true labels of these points
        labels = y_true[indices]
        # count most common label
        most_common_count = Counter(labels).most_common(1)[0][1]
        total_correct += most_common_count
    return total_correct / len(y_true)
y_true = ["Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "No", "No"]
y_pred = [1, 0, 2, 2, 1, 1, 1, 2]
purity = purity_score(y_true, y_pred)
print("Purity:", purity)
```

4. LLM-Era Clustering

- The resulting clusters using traditional approaches lack interpretability, as clustering models typically do not produce meaningful labels for the groups. These challenges make traditional clustering approaches less flexible and efficient, particularly when applied to diverse, large-scale text datasets [6].
- **LLM-based text clustering** groups documents by semantic similarity using large language models (LLMs), typically via dense embeddings rather than sparse bag-of-words features. Compared with traditional approaches, it captures deeper meaning, paraphrases, and context.
- Core Steps:
 1. **Embedding**
Convert each document into a dense vector using an LLM (e.g., BERT based sentence or document embeddings).
 2. **(Optional) Dimensionality Reduction**
Reduce noise and improve clustering stability (commonly UMAP).
 3. **Clustering**
Group vectors using algorithms such as K-means, Hierarchical clustering, or density-based methods like HDBSCAN.
 4. **Cluster Interpretation / Labeling**
Use an LLM to summarize or name each cluster based on representative texts.

Prompt-based generative grouping

20

- Ask an LLM to iteratively group texts or assign cluster IDs via prompts.
- API-based LLMs do not provide direct access to their internal embeddings, limiting their adaptability in clustering applications [6].

Prompt Example:

You are performing generative clustering.

Group the texts below into 3 clusters based on semantic similarity.

For each cluster: Provide a cluster number (1,2,3), A short one-sentence summary, the assigned texts Texts:

T1: "Transformers scale well with large datasets."

T2: "New fitness guidelines emphasize daily physical activity."

T3: "LLMs require significant computational resources."

T4: "Regular exercise reduces cardiovascular disease risk."

T5: "Efficient model training reduces energy consumption." T6: "AI chatbot may give incorrect answers."

Output only a valid Python dictionary cluster number: (assigned texts in a list, one-sentence summary), and nothing else.

ChatGPT output:

```
{
  1: (["T1", "T3", "T5", "T6"], "Texts related to AI, machine learning models, and computational considerations."),
  2: (["T2", "T4"], "Texts focused on fitness and health benefits of regular exercise."),
  3: ([], "No remaining texts outside the above clusters.")
}
```

CRICOS No.00213J

Two-stage LLM driven clustering framework

- It consists of two key stages: label generation stage and text classification stage.
- Label Generation Using LLMs
 - It first divides the entire dataset into some mini-batches since inputting an entire dataset into LLMs is impractical due to context length limitations.
 - For each mini-batch, it generates a prompt p_b and uses it to generate potential labels for each text and save the labels in a python list $L_b = p_b(Instruction, b, labels)$
 - Get all unique labels from all L_b and put them into L_{unique} .
 - To avoid redundant duplication of final clusters caused by the LLM producing different descriptions for the same label, it further prompts the LLM to merge labels with similar expressions by using a merge prompt p_m :
$$L = p_m(Instruction_of_Merge, L_{unique})$$
 - For each input instance, it prompts the LLM to assign a label from the generated labels L .

Example prompt to general lables

You are a text classification system.

Task: Assign one label from the following set:

{Sports, Politics, Technology, Health, Entertainment}

Text 1: “Apple announced a new chip architecture that significantly improves AI performance on mobile devices.”

Text 2: “The use of machine learning models in patient diagnosis raises concerns about data privacy and ethical responsibility.”

Generate exactly one label per text. Output only a valid Python list of labels and nothing else.

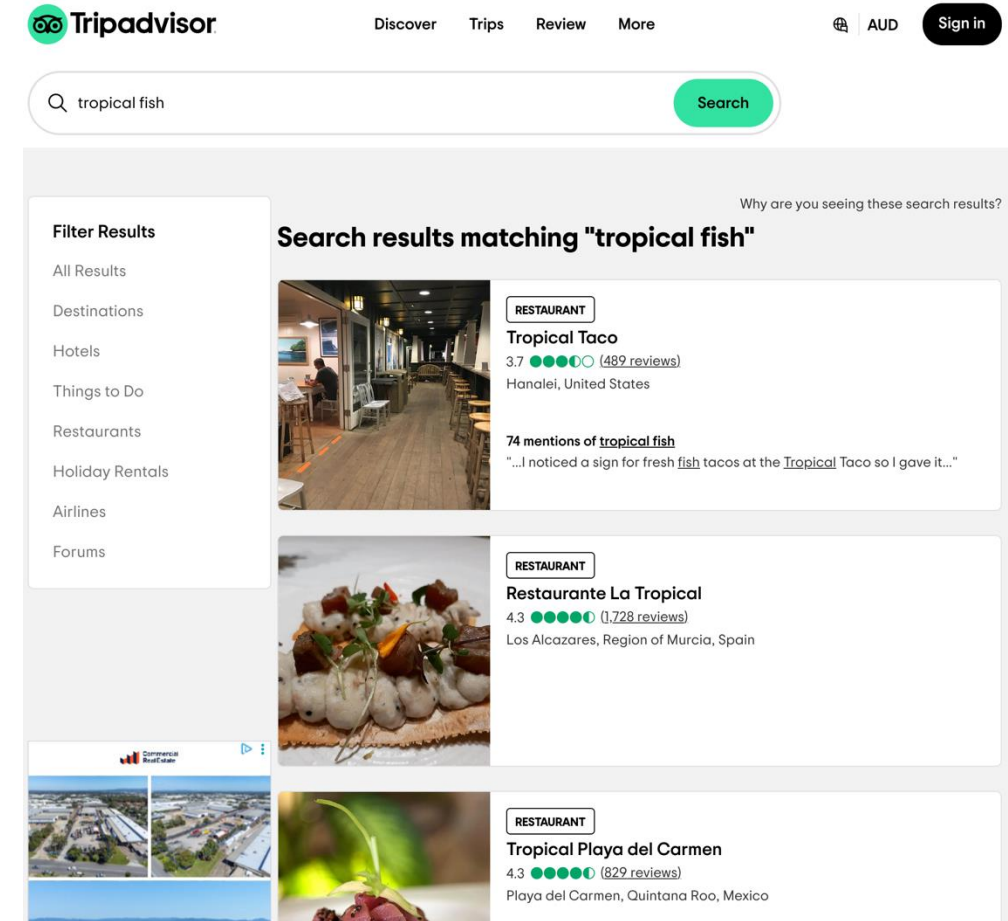
” Ask ChatGPT

["Technology", "Health"]

5. Recommender Systems

23

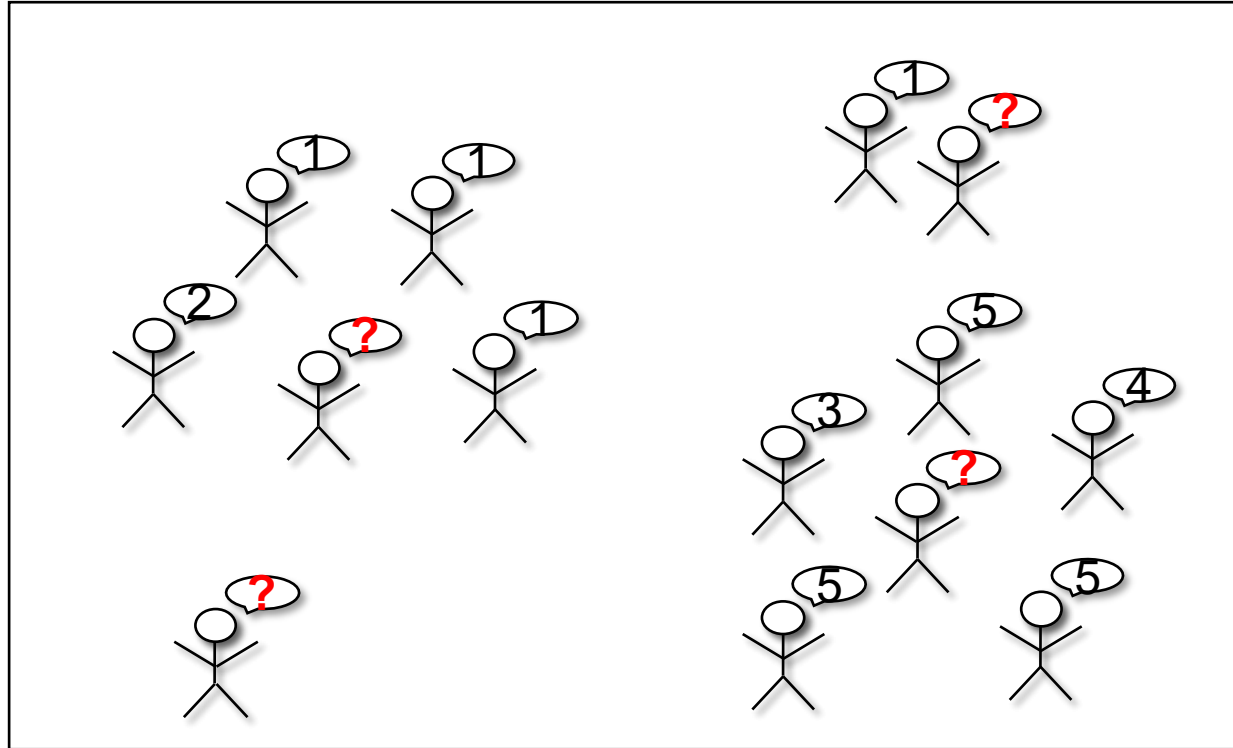
- Recommender systems are tools designed to enhance the consumption of items and streamline the process for customers to find their preferred items.
- Examples
Amazon.com, Netflix, TripAdvisor
- In static and adaptive filtering, users and their profiles are assumed to be independent of each other.
- However, in real world, similar users are likely to have similar preferences.
- There are three main types of Recommender Systems classified based on how the **ratings** are filtered. These include Content-based filtering, Collaborative filtering, and Hybrid filtering
- **Collaborative filtering** exploits relationships between users to improve how items (documents) are matched to users (profiles).
- News recommendation is a good example of use NLP techniques.



Inputs and output of Collaborative filtering

- In recommender systems, user/item profiles are established by analyzing user engagements with items in the system. The ratings of items are also a key data source.
- **Inputs**
 - $(user, item, rating)$ tuples for items that the user has explicitly rated
 - Typically represented as a user-item matrix.
- **Output**
 - $(user, item, rating)$ tuples for items that the user has not rated
 - Can be thought of as filling in the missing entries of the user-item matrix.
- Most algorithms infer missing ratings (the sparsity problem) based on the ratings of similar users.

Example 3. Users and their ratings for items



- Users and their ratings for an item (e.g., the textbook).
- Users with question marks above their heads have not yet rated the item.
- The goal of the recommender system is to fill in these question marks.

Rating Method - Using User Clusters

- This rating method uses clustering technique to find groups of similar users.
- Measure user/user similarity using rating correlation (Pearson Correlation) for clustering users:

$$\frac{\sum_{i \in I_u \cap I_{u'}} (r_u(i) - \bar{r}_u) \cdot (r_{u'}(i) - \bar{r}_{u'})}{\sqrt{\sum_{i \in I_u \cap I_{u'}} (r_u(i) - \bar{r}_u)^2 \sum_{i \in I_u \cap I_{u'}} (r_{u'}(i) - \bar{r}_{u'})^2}}$$

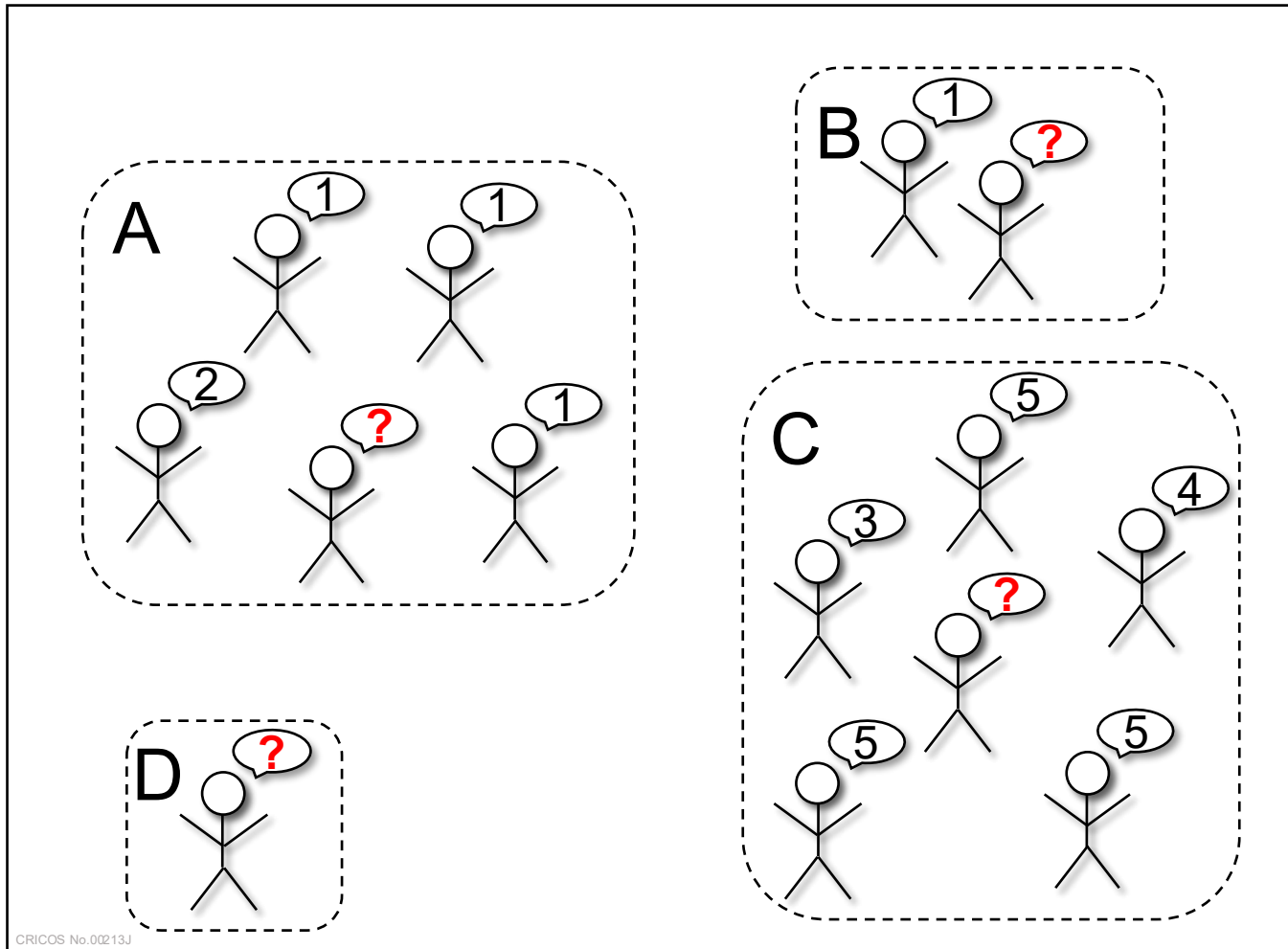
- Where $r_u(i)$ is user u 's rating of item i ; I_u and $I_{u'}$ are the sets of items that user u and user u' judged, respectively. $\bar{r}_u$ means the u 's average rating.
- The predicted rating to an unseen item i is to use average rating of **other users** within the same cluster:

$$\hat{r}_u(i) = \frac{1}{|Cluster(u)|} \sum_{u' \in Cluster(u)} r_{u'}(i)$$

Example 3 cont.

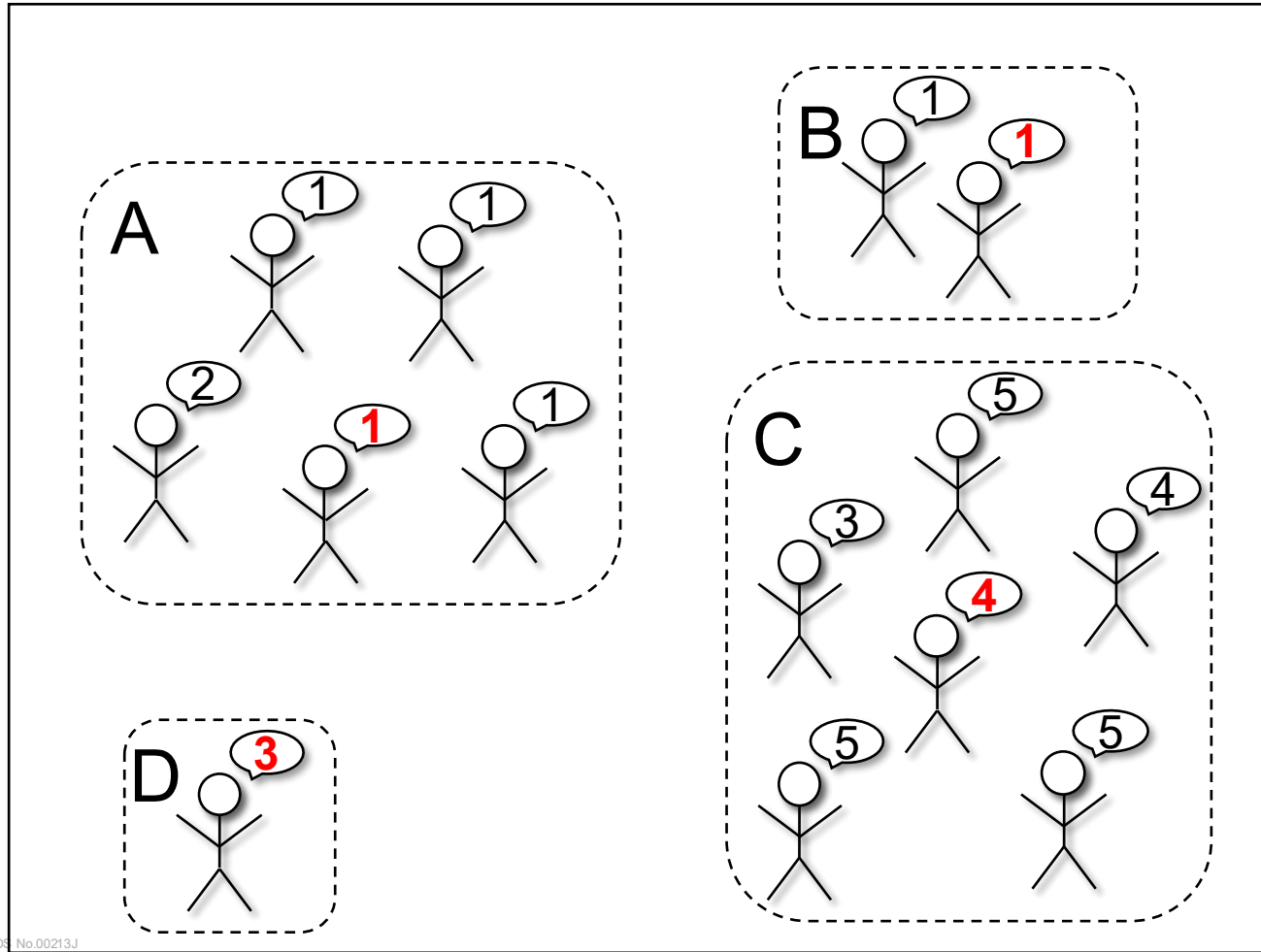
Cluster-Based Collaborative Filtering

27



- A clustering algorithm can be used to group users into 4 clusters: *A*, *B*, *C* and *D*.
- In each cluster, there is a single user who has not judged the item.

Example 3 cont.



- Each unjudged item is assigned automatic rating based on the ratings of similar users.
- By using the rating method 1, the unseen ratings are predicted by using the average rating of **other users** within the same cluster.
 - E.g., for the unseen rating in C, the average is $(3+4+5+5+5)/5 = 4.4 \approx 4$

Rating Method - using nearest neighbors

- Similar to K -nearest neighbor algorithm, we can also infer ratings based on nearest neighbors.
 - Weight ratings of nearest neighbor according to similarity.
- It predicts user u 's rating of item i by first calculating the user's average item rating. Then, for every user u' in u 's nearest neighbourhood $N(u)$, (*rating - average rating*) is weighted by $sim(u, u')$ and added into the predicted value:

$$\hat{r}_u(i) = \bar{r}_u + \frac{1}{\sum_{u' \in N(u)} sim(u, u')} \sum_{u' \in N(u)} sim(u, u')(r_{u'}(i) - \bar{r}_{u'})$$

- where $sim(u, u')$ is the similarity of user u and u' , and $N(u)$ is the set of user u 's nearest neighbours.
- Best to use (*rating - average rating*) rather than just *rating* for other user u' because ratings are relative (i.e., Different users may use different grading standards), not absolute.

Example of users, items, and ratings for a collaborative filtering system

- Users and Items

```
U = [0, 1, 2, 3, 4, 5] # 6 users
I = [0, 1, 2, 3, 4] # 5 items
```

- User–Item Rating Matrix (Rvs) or user rating vectors

```
Rvs = [
  [1, 2, 1, 0, 0], # User 0
  [0, 1, 1, 1, 1], # User 1
  [2, 1, 3, 5, 2], # User 2
  [1, 0, 2, 0, 3], # User 3
  [0, 2, 3, 4, 5], # User 4
  [0, 1, 3, 0, 0]  # User 5
]
```

- Each row represents a user
- Each column represents an item
- Each value represents the rating (e.g., 0–5 scale)
- A value of 0 means the item has not been rated

Evaluating Collaborative Filtering

- Standard metrics, such as precision are too strict for evaluating recommender systems
- Want to quantify how different predicted rating are from actual ratings

- Absolute error

$$ABS = \frac{1}{|\mathcal{U}||\mathcal{I}|} \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}} |\hat{r}_u(i) - r_u(i)|$$

- Mean squared error

$$MSE = \frac{1}{|\mathcal{U}||\mathcal{I}|} \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}} (\hat{r}_u(i) - r_u(i))^2$$

- The biggest difference between absolute error and mean squared error is that mean squared error penalizes incorrect predictions more heavily, since the penalty is squared.

6. LLMs for News Recommendation

- The rise of personalized news recommendation systems, like Google News and Microsoft News, has revolutionized how people access and consume news by making it more tailored to individual preferences.
- LLMs leverage extensive linguistic and world knowledge from large-scale corpora to understand and generate human-like language, making them valuable for aligning news content with user preferences and enhancing explainability in recommendation systems.
 - Cons: LLM-based Prompting strategies have demonstrated effectiveness in complex tasks but do not surpass well-trained deep neural methods in performance.
 - Few works focus on studying the generation of explanations for recommendations and their evaluation due to the lack of ground truth.
 - Pros: The nature of LLMs makes them well-suited for generating explanations for recommendations, e.g., ChatRec
- RecPrompt [7] is a self-tuning prompting framework specifically designed for news recommendations.
 - RecPrompt operates through an iterative bootstrapping process involving two LLMs: a news recommender and a prompt optimizer.
 - The recommender generates recommendations with text prompts, and these are then fed into the optimizer, which refines the prompt based on the instructions, enhancing the prompt's alignment with user preferences and topics of interest.

News Recommender

- For a user u as, let $\mathbf{H}_u$ be the history of u , a set of news articles nw_i .
- The candidate news is defined as $\mathbf{D}_u = \{(nw_j, y_j)\}$, with each news article accompanied by a label $y_j \in \{0, 1\}$.
 - A value of 1 indicates that the user is interested in the candidate news, while 0 indicates no interest.
- The objective is to learn a news recommender, and for any unseen user u' , the output is a ranked list of the candidate news $\mathbf{R}_{u'}$.

Input-Output (IO) prompting template:

You serve as a personalized news recommendation system.

Input Format

User's History News

$\{\text{history}\}$

Candidate News

$\{\text{candidate}\}$

Output Format

Rank candidate news based on the user's history news in the format: "Ranked news: <START>C#, C#,..., C#<END>".

Example prompt for news recommendation

You serve as a personalized news recommendation system.

Input Format

User's History News

\${"Large Language Models and the Future of Search", "AI Regulation and Data Privacy in the EU", "Scalable Transformer Architectures"}

Candidate News

\${C1: "Australia Announces New AI Safety Framework", C2: "Breakthrough in Protein Folding Using Deep Learning", C3: "Celebrity Influencer Trends in 2026", C4: "Machine Learning Techniques for Secure Data Sharing"}

Output Format

Rank candidate news based on the user's history news in the format: "Ranked news: <START>C#: Reason, C#: Reason,..., C#: Reason<END>".

” Ask ChatGPT

Ranked news: <START>C1: Directly aligns with the user's interest in AI regulation and governance, especially following prior reading on AI regulation and data privacy, C4: Strongly matches the user's focus on machine learning and data privacy through secure data sharing techniques, C2: Related to deep learning but oriented toward scientific discovery rather than the user's primary interests in AI systems and policy, C3: Unrelated to the user's demonstrated interests in AI, machine learning, and regulation<END>

Prompt Optimizer

- The task of Prompt Optimizer is to generate a better recommendation prompt by taking an optimization prompt as input.
- This prompt contains a refinement instruction, the recommendation prompt for a user, the ranked news sequence and the explanation, as well as the ground truth, *best template recorded by the monitor*, and an observation instruction, where the monitor evaluates the recommender's performance using the templates generated by the optimizer.
- The refinement and observation instructions are crucial in guiding the optimizer to a specific direction.
- The refinement instruction typically starts with a sentence like "You should generate an improved template instruction based on the provided information." at the beginning.
- The observation instruction is designed to focus on topics for the news recommendation task. An example observation instruction could be:
 - You should focus on how well the recommender's response aligns with the user's click behavior by examining if the topics from the user's news history and candidate news are accurately summarized and matched to the user's interests. Specifically, evaluate the clarity of topics in the recommender's answer, review the task description for adequacy, and check the detail in the recommendation process to ensure it reflects an analysis and summary of user-interest topics.

Example of Ask ChatGPT again:

Refine the previous prompt. You should generate an improved template instruction based on the provided information.

You should focus on how well the recommender's response aligns with the user's history and candidate news are accurately summarized and matched to the user's interests. Specifically, evaluate the clarity of topics in the recommender's answer, review the task description for adequacy, and check the detail in the recommendation process to ensure it reflects an analysis and summary of user-interest topics.

✦ Refined Prompt Template: Personalized News Recommendation (Analysis-Focused)

System Role

You are an expert personalized news recommendation system. Your task is to rank candidate news articles by how well they align with the user's demonstrated interests, based strictly on their reading history. Your recommendations must reflect clear topic matching, accurate content understanding, and explicit relevance reasoning.

◆ Input Format

User's Reading History

A list of news articles previously read by the user, representing their core interests and preferred topics.

```
arduino
{
  "Large Language Models and the Future of Search",
  "AI Regulation and Data Privacy in the EU",
  "Scalable Transformer Architectures"
}
```

Candidate News Articles

A set of candidate articles to be evaluated and ranked.

```
css
{
  C1: "Australia Announces New AI Safety Framework",
  C2: "Breakthrough in Protein Folding Using Deep Learning",
  C3: "Celebrity Influencer Trends in 2026",
  C4: "Machine Learning Techniques for Secure Data Sharing"
}
```

◆ Task Instructions

1. Interest Extraction

- Identify the **main topics** and **themes** reflected in the user's reading history (e.g., large language models, AI governance, data privacy, scalable ML systems).

2. Candidate Evaluation

- For each candidate article:
 - Clearly summarize its **primary topic** in relation to the user's interests.
 - Assess how strongly it aligns with the extracted themes.
 - Penalize articles that are off-topic or unrelated.

3. Ranking Criteria

- Rank articles by **descending relevance** to the user's interests.
- Favor candidates that:
 - Directly match multiple interest themes
 - Are conceptually close (e.g., AI policy ↔ AI safety, ML systems ↔ secure data sharing)
- Deprioritize articles with weak, indirect, or no relevance.

4. Reasoning Quality

- Provide **concise but explicit justifications** showing:
 - Topic overlap with user history
 - Why higher-ranked items are more relevant than lower-ranked ones
- Avoid vague explanations; reasoning must reflect **analysis and summarization** of user-interest topics.

◆ Output Format (Strict)

```
powershell
Ranked news: <START>
C#: Reason,
C#: Reason,
C#: Reason,
C#: Reason
<END>
```

- Reasons must be **topic-focused, comparative, and grounded in the user's history**.
- Do not include any text outside the specified format.

References

- [1] Chapter 4, chapter 9 & chapter 10 in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.
- [2] Jansen, B. J. (2009). Understanding user-web interactions via web analytics. *Synthesis Lectures on Information Concepts, Retrieval, and Services*, 1(1), 1-102.
- [3] M. Perkowitz and O. Etzioni, “Adaptive web sites: an AI challenge”, 1997, *IJCAI’97*, 16-23.
- [4] Xue, X., Jeon, J., & Croft, W. B. (2008). Retrieval models for question and answer archives. In *SIGIR ’08*, pp. 475–482.
- [5] Bengio, Yoshua, Réjean Ducharme, and Pascal Vincent. "A neural probabilistic language model." *Advances in neural information processing systems* 13 (2000). <https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf>
- [6] Chen Huang and Guoxiu He. 2025. Text Clustering as Classification with LLMs. In *Proceedings of the 2025 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region (SIGIR-AP 2025)*, pp. 374–384, <https://doi.org/10.1145/3767695.3769519>
- [7] D. Liu, et al., RecPrompt: A Self-tuning Prompting Framework for News Recommendation Using Large Language Models, in *CIKM '24: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, Pages 3902 – 3906, <https://doi.org/10.1145/3627673.3679987>